

图形学渲染大作业：路径追踪

苏煜宸 2022013267

1 Whitted Ray

功能点：有限递归光线追踪，折射、漫反射、镜面反射，阴影

```
function raytrace(ray, depth):
if depth > MAX_DEPTH:
    return background_color

hit = FindNearestIntersection(ray)
if hit == null:
    return background_color

color = AmbientTerm(hit)

for each light in scene:
    shadow_ray = ComputeShadowRay(hit.position, light)
    if not IsInShadow(shadow_ray):
        color += DiffuseTerm(hit, light)
        color += SpecularTerm(hit, light, ray)

if hit.material.reflective:
    reflected_ray = ComputeReflectedRay(hit, ray)
    reflected_color = TraceRay(reflected_ray, depth + 1)
    color += hit.material.reflectivity * reflected_color

if hit.material.transmissive:
    refracted_ray = ComputeRefractedRay(hit, ray)
    if refracted_ray is not null:
        refracted_color = TraceRay(refracted_ray, depth + 1)
        color += hit.material.transparency * refracted_color
```

```
return color
```

Whitted Ray 结果如图 1所示。

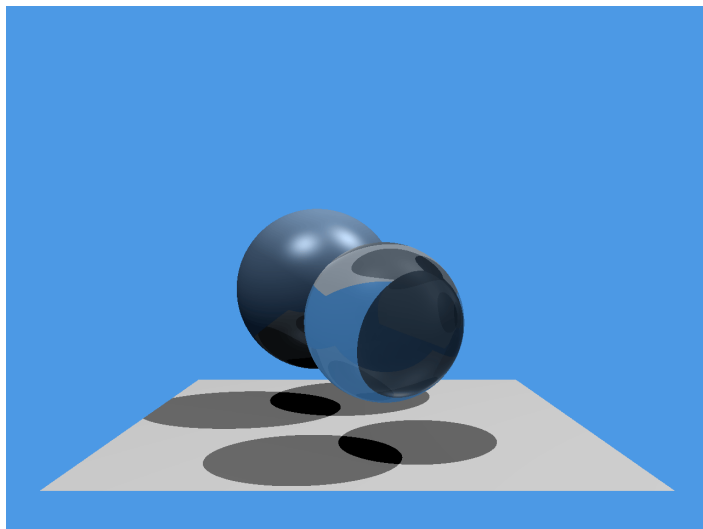


图 1: Whitted Ray

2 法向插值

设三角形的三个顶点为 A, B, C ，对应的法向为 $\vec{N}_A, \vec{N}_B, \vec{N}_C$ 。对于三角形内部任意一点 P ，其重心坐标为 (α, β, γ) ，满足：

$$\alpha + \beta + \gamma = 1, \quad \alpha, \beta, \gamma \geq 0$$

则点 P 的插值法向为：

$$\vec{N}_P = \alpha \vec{N}_A + \beta \vec{N}_B + \gamma \vec{N}_C$$

并进行归一化。结果如图 2所示。

3 路径追踪

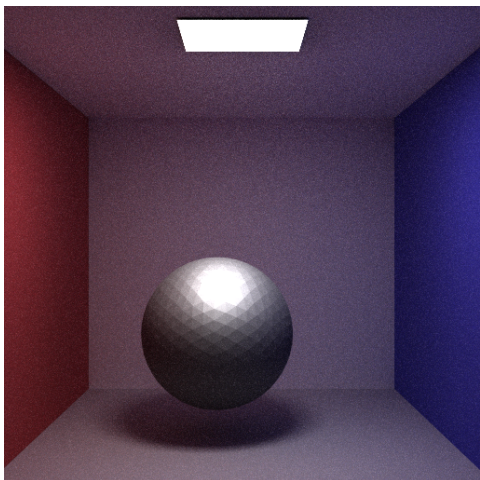
基本功能点：面光源，无穷递归，漫反射，镜面反射，折射，glossy BRDF，NEE 带有 NEE 和不带有 NEE 的对比如 3所示。

最终路径渲染效果图如下，其中前方的小球使用了 glossy 材质，左侧钻石和右侧大球使用折射 BRDF 采样，后侧兔子使用反射 BRDF 采样。

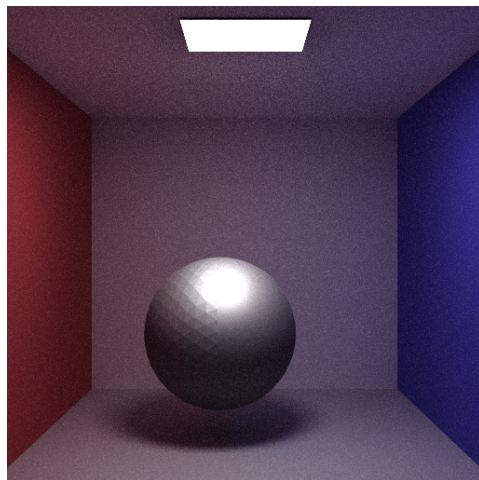
附加功能：

Algorithm 1 路径追踪 (Path Tracing) 伪代码

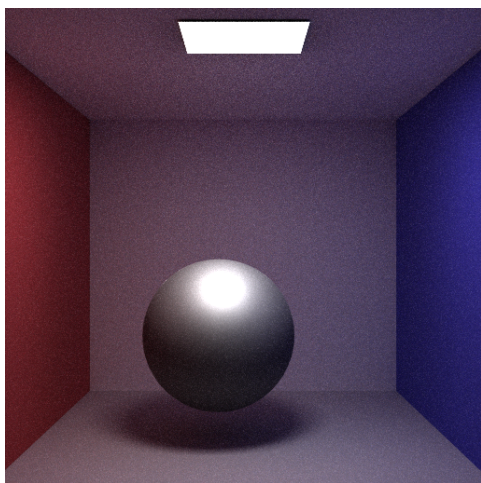
```
1: function PATHTRACENE(ray, depth)
2:   if depth > maxDepth then
3:     return 0
4:   end if
5:   hit  $\leftarrow$  IntersectScene(ray)
6:   if not hit then
7:     return backgroundColor
8:   end if
9:   x  $\leftarrow$  hit.position
10:  n  $\leftarrow$  hit.normal
11:  material  $\leftarrow$  hit.material
12:  emission  $\leftarrow$  material.emission
13:  incomingDir  $\leftarrow$   $-ray.direction$ 
14:  newDir  $\leftarrow$  SampleDirection(material, n)
15:  brdf  $\leftarrow$  EvaluateBRDF(material, incomingDir, newDir, n)
16:  pdf  $\leftarrow$  PDF(material, incomingDir, newDir, n)
17:  newRay  $\leftarrow$  Ray(x +  $\epsilon \cdot newDir$ , newDir)
18:  L  $\leftarrow$  RADIANCE(newRay, depth + 1)
19:  return emission +  $\frac{brdf \cdot L \cdot \cos \theta}{pdf}$ 
20: end function
```



(a) 未插值

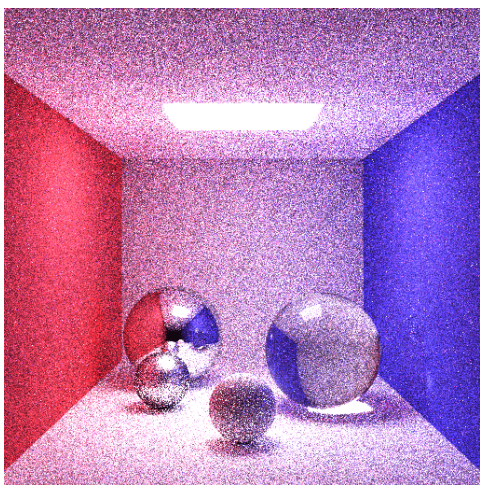


(b) 一半插值, 一半未插值

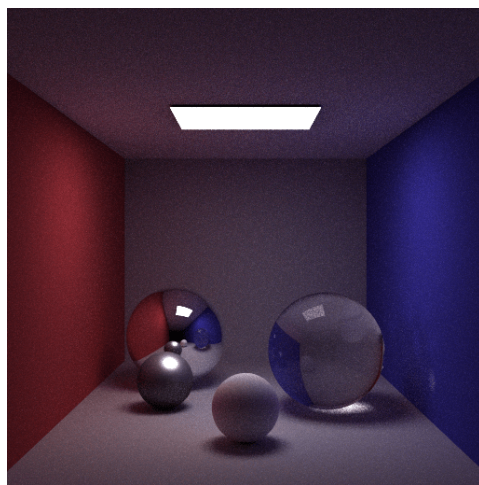


(c) 全部插值

图 2: 法向插值结果图



(a) spp = 100, 不带有 NEE



(b) spp = 100, 带有 NEE

图 3: 带有 nee 和不带有 nee 对比

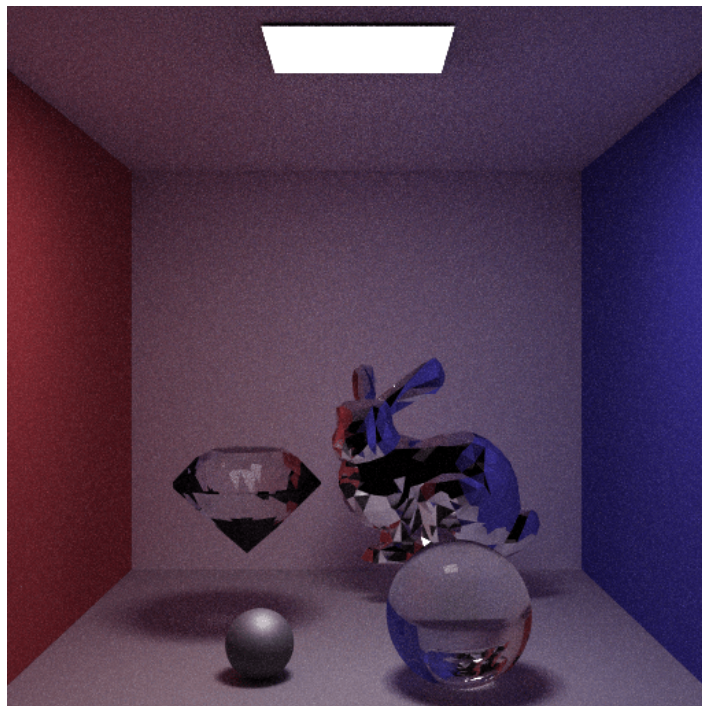


图 4: 路径渲染

材质贴图：使用文件中的 vt，通过重心坐标获得插值结果

cos-weighted 采样：采用 cos 分布函数进行漫反射的采样

多线程加速：openmp

4 高级功能

4.1 复杂网格和加速求交

我们使用 AABB 构建 Bounding Box，并构建 kdtree 进行求解，其原理如下

加入附加功能后的结果如 5 所示。其中花朵有 8 万个顶点和约 20 万个三角形，使用 spp=100 进行求交点大约需要 10 分钟。

5 参数曲面求交

设 Bézier 曲线由控制点 $P_0, P_1, \dots, P_n$ 定义，其表达式为 $B(t) = \sum_{i=0}^n B_i^n(t) P_i$ ，其中 Bernstein 基函数为 $B_i^n(t) = \binom{n}{i} (1-t)^{n-i} t^i$ 。使用 de Casteljau 算法可以递归求得曲线在参数 t 下的位置，方法是：设初始点为 $P_i^{(0)} = P_i$ ，然后进行逐层线性插值，递推关系为 $P_i^{(k)}(t) = (1-t)P_i^{(k-1)}(t) + tP_{i+1}^{(k-1)}(t)$ ，当 $k = n$ 时，最终得到 $B(t) = P_0^{(n)}(t)$ ，即为曲线上对应点的位置。对于切向（导数），Bézier 曲线的一阶导数可表示为 $B'(t) = n \sum_{i=0}^{n-1} (P_{i+1} - P_i) B_i^{n-1}(t)$ ，或者可构造新的控制点集 $D_i = n(P_{i+1} - P_i)$ ，再用 de Casteljau 算法计算该导数曲线在 t 处的位置作为切向。B 样条曲线由控制点 $P_0, P_1, \dots, P_n$

Algorithm 2 BuildKDTree(objects, bbox, depth)

```
1: if  $depth \geq max\_depth$  or  $|objects| \leq max\_objects\_per\_leaf$  then
2:   return LeafNode with objects and bbox
3: end if
4:  $axis \leftarrow depth \bmod 3$ 
5:  $centers \leftarrow$  [center of obj's bbox along axis for obj in objects]
6:  $split\_pos \leftarrow median(centers)$ 
7:  $left\_box \leftarrow$  bbox with  $max[axis] = split\_pos$ 
8:  $right\_box \leftarrow$  bbox with  $min[axis] = split\_pos$ 
9:  $left\_objects \leftarrow$  [obj for obj if obj intersects left_box]
10:  $right\_objects \leftarrow$  [obj for obj if obj intersects right_box]
11: if splitting is ineffective then
12:   return LeafNode with objects and bbox
13: end if
14:  $node \leftarrow$  new KDNode
15:  $node.split\_axis \leftarrow axis$ 
16:  $node.split\_pos \leftarrow split\_pos$ 
17:  $node.left \leftarrow BUILDKDTREE(left\_objects, left\_box, depth + 1)$ 
18:  $node.right \leftarrow BUILDKDTREE(right\_objects, right\_box, depth + 1)$ 
19: return node
```

Algorithm 3 IntersectKDTree(*node*, *ray*, *hit*, *tmin*)

```
1: if ray misses node.bbox then
2:   return false
3: end if
4: if node is leaf then
5:   for all obj  $\in$  node.objects do
6:     if ray intersects obj then
7:       update hit record
8:     end if
9:   end for
10:  return whether hit was found
11: end if
12: axis  $\leftarrow$  node.split_axis
13: split_pos  $\leftarrow$  node.split_pos
14: origin  $\leftarrow$  ray.origin[axis]
15: direction  $\leftarrow$  ray.direction[axis]
16: if origin < split_pos or (origin = split_pos and direction  $\leq$  0) then
17:   first  $\leftarrow$  node.left
18:   second  $\leftarrow$  node.right
19: else
20:   first  $\leftarrow$  node.right
21:   second  $\leftarrow$  node.left
22: end if
23: hit_first  $\leftarrow$  INTERSECTKDTREE(first, ray, hit, tmin)
24: if hit_first and hit.t < distance to split plane then
25:   return true
26: end if
27: hit_second  $\leftarrow$  INTERSECTKDTREE(second, ray, hit, tmin)
28: return hit_first or hit_second
```

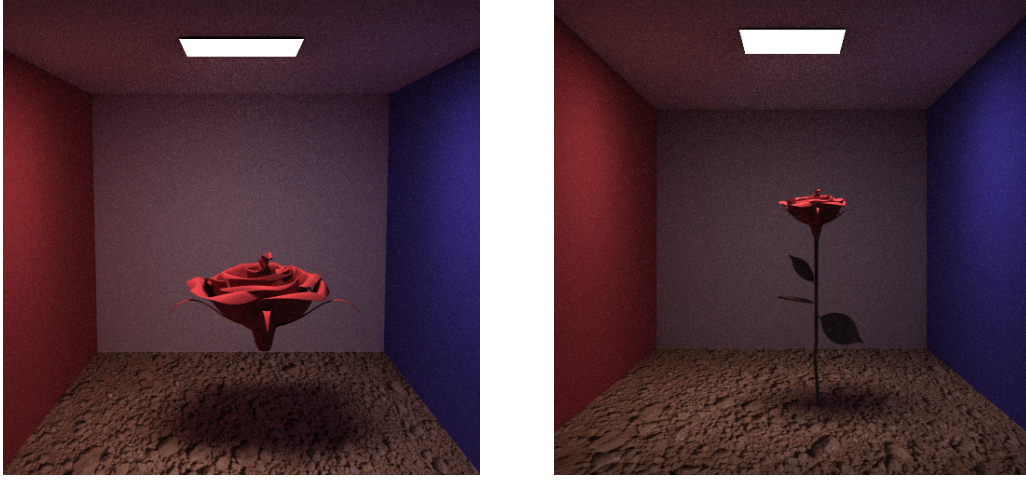


图 5: 路径渲染附加功能

和节点矢量 $U = \{u_0, u_1, \dots, u_{n+p+1}\}$ 定义，其曲线表达式为 $C(t) = \sum_{i=0}^n N_{i,p}(t)P_i$ ，其中 $N_{i,p}(t)$ 是 p 次 B 样条基函数。为计算在给定参数 t 下的点位置，使用 de Boor 算法：首先确定使 $t \in [u_k, u_{k+1})$ 的区间索引 k ，然后初始化 $d_i^{(0)} = P_{k-p+i}$ ，接着进行 p 次递推，递推公式为 $\alpha_i^{(r)} = \frac{t - u_{i+k-p}}{u_{i+1+k-r} - u_{i+k-p}}$ ，以及 $d_i^{(r)} = (1 - \alpha_i^{(r)})d_{i-1}^{(r-1)} + \alpha_i^{(r)}d_i^{(r-1)}$ ，最终得到曲线点 $C(t) = d_p^{(p)}$ 。曲线切向的计算可用导数形式 $C'(t) = \sum_{i=0}^n N'_{i,p}(t)P_i$ ，也可以将控制点做差分构造一阶导数曲线，并再次使用 de Boor 算法计算其值以获得切向。

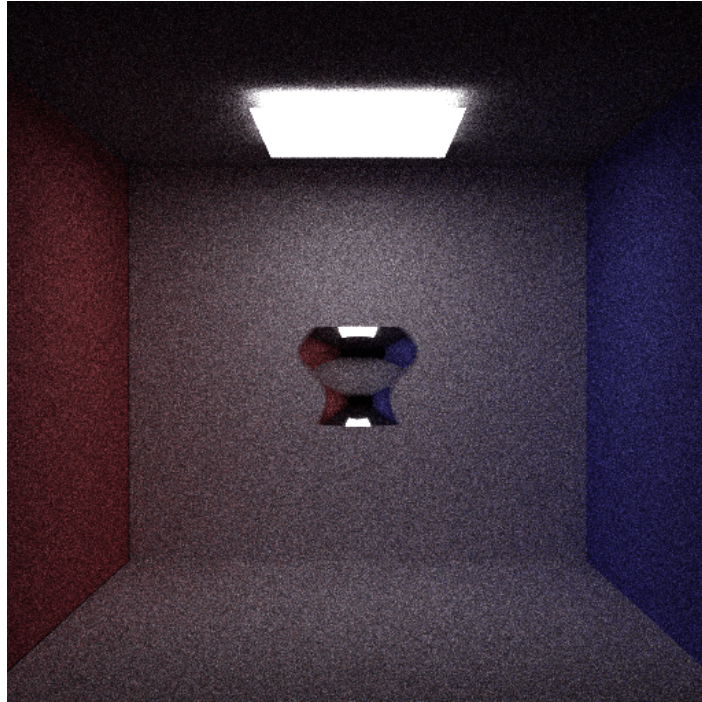


图 6: 参数曲面